VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY
UNIVERSITY OF SCIENCE

COMPUTATIONAL THINKING
SMART TOURISM SYSTEM PROJECT

Summoner's Rift Transportation Web Application

PROJECT REPORT

Class: 24C11
Group: GROUP 2

December 2025

Contents

List of Figures

List of Tables

Team Members & Task Allocation

Student ID	**Member**	**Role**	**Primary Responsibilities**
24127021	Nguyễn Thành Đạt	Routing & SOS Mapping	Integration of TomTom APIs, route computation, and visualization of navigation and emergency routes
24127041	Hoàng Trung Hiếu	Bus Mapping & Recommendation Engine	Design and implementation of pathfinding algorithms, development of the trip recommendation system
24127298	Lương Hưng Phát	Traffic Congestion Analysis	Real-time traffic congestion detection and analysis from camera feeds
24127188	Nguyễn Minh Khoa	Data Acquisition	Collection and preprocessing of real-time traffic camera images
24127430	Nguyễn Anh Khôi	Database Engineering	Database schema design, optimization, and indexing for high-performance queries
24127373	Nguyễn Tấn Hiệu	UI/UX Design & System Integration	Frontend development, user experience design, and cross-module API integration

Table 1: Team roles and responsibilities

Task Completion Assessment

Student ID	Member	Completed Tasks	Completion (%)
24127021	Nguyễn Thành Đạt	TomTom API integration, geocoding, route computation, route visualization, SOS reporting backend & frontend	100%
24127041	Hoàng Trung Hiếu	Bus data ETL, A* pathfinding for bus routes, LLM-based trip recommendation, fuzzy place matching, itinerary map visualization	100%
24127298	Lương Hưng Phát	YOLO vehicle detection, detection aggregation pipeline, congestion classification, UI integration	100%
24127188	Nguyễn Minh Khoa	Traffic camera scraping scripts, camera metadata, data preprocessing	100%
24127430	Nguyễn Anh Khôi	MySQL schema design, connection pooling, query optimization, indexing strategy	100%
24127373	Nguyễn Tấn Hiệu	React UI components, Goong Maps integration, authentication flow, responsive styling	100%

Table 2: Job completion assessment per team member

Project Repository

The source code and additional resources for this project are available at:

- **GitHub**: `https://github.com/AughustN/All-In-One-Mobility-Web-Platform`
- **Google Drive**: `https://drive.google.com/drive/folders/1f4w6P_2RpLogz-z6AZYbHj-z8j_U3TTI?hl=vi`

Chapter 1

Problem Analysis

1.1 Contextual Background

Exploring a large city should feel intuitive and rewarding, especially for visitors with limited time. However, many travelers struggle not because the city lacks attractions or transportation options, but because moving between places is unnecessarily complicated. Visitors are often unsure which route is truly the best, how long a journey will realistically take, or whether a chosen option will still make sense once traffic, transfers, and waiting time are taken into account.

This lack of clarity creates a series of challenges that directly undermine the tourism experience:

- **Too many choices, not enough clarity:** Tourists are presented with multiple transport options, yet lack confidence in knowing which one will actually get them to their destination faster, cheaper, and with less hassle.
- **Time lost to planning instead of exploring:** Valuable travel time is spent comparing routes, checking schedules, and adjusting plans on the go rather than enjoying the city itself.
- **Stress in unfamiliar environments:** Navigating public transport systems, transfers, and traffic conditions can feel intimidating, particularly for first-time visitors.
- **Late reactions to delays and congestion:** Travelers often become aware of problems only after they are already affected, with limited visibility into better alternatives.
- **Fewer experiences, lower satisfaction:** To avoid uncertainty, visitors tend to play it safe—skipping destinations, shortening itineraries, or choosing costly transport options—ultimately reducing the overall quality of their trip.

Instead of confidently exploring the city, travelers become reactive and risk-averse. For short stays in particular, this friction directly limits how much of the city can be experienced and significantly diminishes overall travel satisfaction.

1.2 Project Goal

The primary objective of this project is to develop a comprehensive, AI-featured Tourism System. This platform aims to bridge the gap between complex urban data and user-friendly travel assistance by providing:

1. **Intelligent Trip Planning:** A Generative AI solution that creates personalized itineraries by fuzzy-matching user preferences against a verified database of locations.
2. **Optimized Public Transit Routing:** A custom A* pathfinding algorithm that accounts for real-time traffic conditions to suggest the most efficient bus routes.
3. **Live Congestion Monitoring:** A computer vision pipeline utilizing YOLO11L to detect vehicle density from over 500 traffic cameras in real-time.
4. **Seamless Integration:** A unified web interface that aggregates all these features into a single, intuitive dashboard.

1.3 Use Case Scenarios

Scenario 1: The Automated Concierge A traveler lands at Tan Son Nhat Airport with a simple request: "I have 2 days in the city, and I want to visit historical museums and try authentic Banh Xeo." The system utilizes its LLM backend to interpret this natural language query, retrieves relevant locations from the database, and generates a logically ordered, geographically optimized itinerary.

Scenario 2: Transit Navigation A user wishes to travel from District 1 to District 3 during rush hour. Instead of providing a static route, the system calculates a dynamic path. It considers walking distances, penalizes routes with multiple transfers (which are psychologically draining), and checks the current congestion levels on the proposed path to ensure the estimated arrival time is accurate.

Scenario 3: Proactive Congestion Avoidance Before leaving their hotel, a user consults the application's map overlay. The system displays road segments color-coded (Blue/Yellow/Red) based on real-time vehicle counts derived from camera feeds, allowing the user to identify and avoid developing bottlenecks before they become stuck in traffic.

Chapter 2

Computational Thinking

2.1 Decomposition

To manage the complexity of this multi-faceted system, we applied the principle of **Decomposition** to break the application down into six distinct, manageable modules. This modular architecture allows for parallel development and easier debugging.

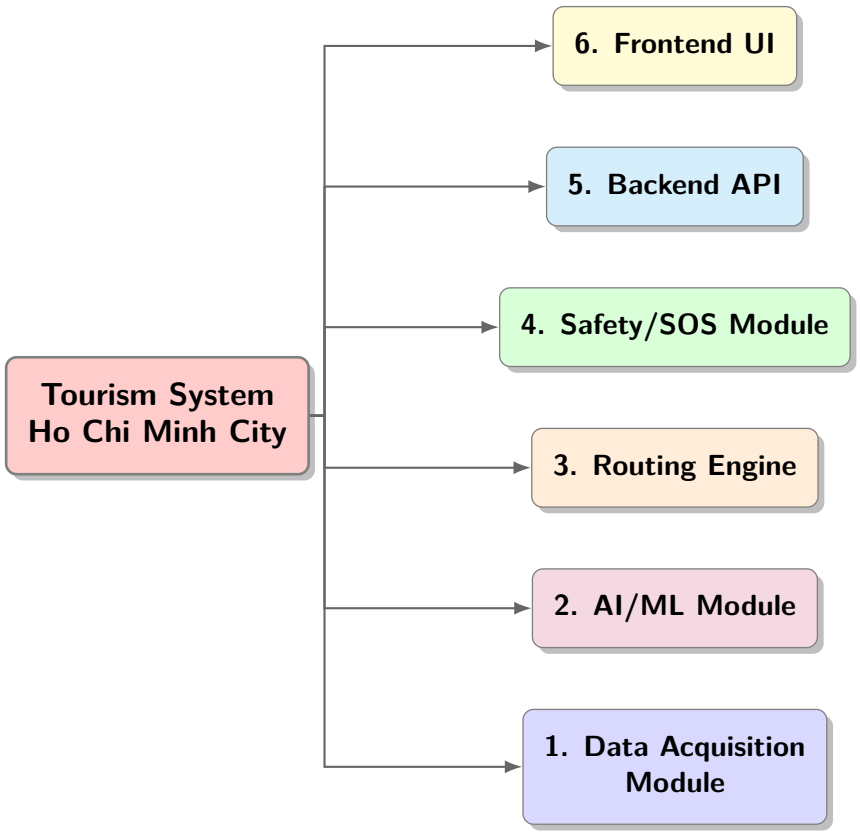
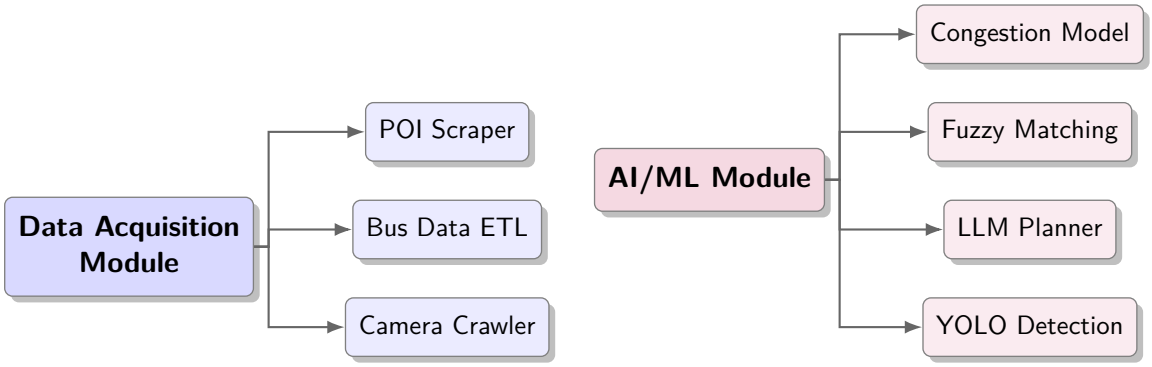


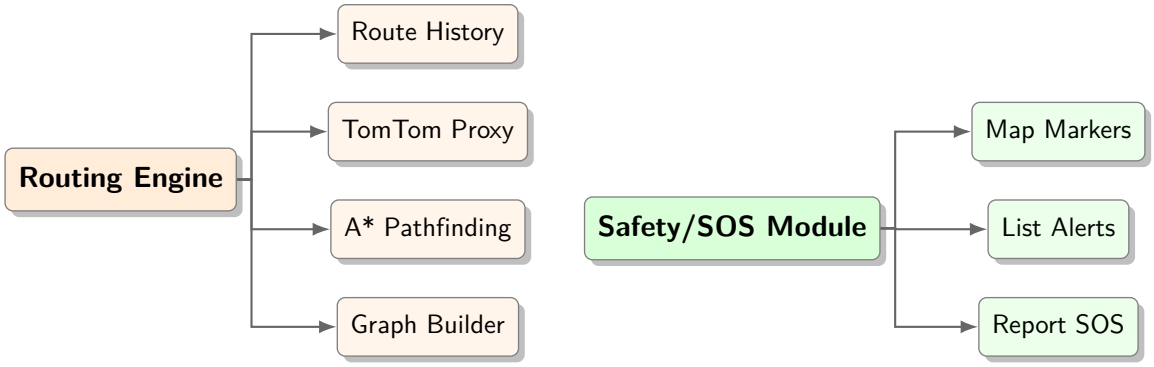
Figure 2.1: System Decomposition Overview

The following figures detail the sub-components of each module:



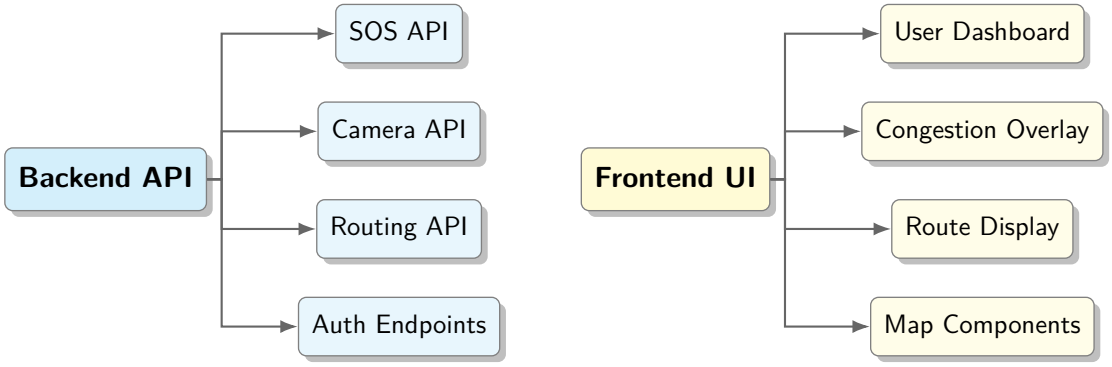
(a) Data Acquisition Module (b) AI/ML Module

Figure 2.2: Decomposition of Data Acquisition and AI/ML Modules.



(a) Routing Engine (b) Safety/SOS Module

Figure 2.3: Decomposition of Routing Engine and Safety/SOS Modules.



(a) Backend API (b) Frontend UI

Figure 2.4: Decomposition of Backend API and Frontend UI Modules.

2.2 Pattern Recognition

Through systematic analysis of the problem domain, several recurring patterns were identified. Recognizing these patterns allowed us to create reusable solutions that significantly reduced code duplication and improved maintainability.

Pattern 1: Geographic Proximity Pattern

Problem Observation: Across multiple features, users consistently need to find nearby resources:

- Bus routing: Find the nearest bus stops to origin/destination
- Traffic monitoring: Find cameras within a geographic region
- Trip planning: Find restaurants, attractions near current location
- SOS alerts: Show incidents near the user's position

Recurring Pattern: All these features require calculating the distance between two GPS coordinates. Rather than implementing distance calculation separately in each module, we identified this as a *cross-cutting concern*.

Solution: Implemented a single `haversine()` function that is reused across all modules:

```python
def haversine(c1, c2):
    """
    Calculate great-circle distance between two GPS points.
    Uses the Haversine formula for spherical geometry.

    Args:
        c1: (latitude, longitude) of point 1
        c2: (latitude, longitude) of point 2
    Returns:
        Distance in meters (float)
    """
    lat1, lon1 = math.radians(c1[0]), math.radians(c1[1])
    lat2, lon2 = math.radians(c2[0]), math.radians(c2[1])
    dlat, dlon = lat2 - lat1, lon2 - lon1
    # Haversine formula
    a = math.sin(dlat/2)**2 + math.cos(lat1)*math.cos(lat2)*math.sin(dlon/2)**2
    return 6371000 * 2 * math.atan2(math.sqrt(a), math.sqrt(1-a))  # Earth radius = 6371km
```

Listing 2.1: Haversine Distance Function - Reused Across All Modules

Application Examples:

```python
# Find nearby bus stops
def nearby_stops(lat, lng, radius=500):
    return [s for s in STOPS if haversine((lat, lng), (s['lat'], s['lng'])) <= radius]

# Filter cameras by location
nearby_cameras = [c for c in cameras if haversine(user_loc, (c['lat'], c['lon'])) < 2000]

# Find places near route
matched = [p for p in places if haversine(route_point, (p['lat'], p['lng'])) < 100]
```

Listing 2.2: Pattern Application in Different Modules

Pattern 2: Data Caching Pattern

Problem Observation: Several operations involve loading and processing large datasets:

- Bus network graph: 4,343 stops, 10,000+ edges

- Place database: Fuzzy matching index for hundreds of locations
- Route information: Pre-computed route metadata

Performance Issue: Without caching, each API request would rebuild these structures, causing:

- Graph construction: 5-10 seconds per request
- Fuzzy index building: 2-3 seconds per request
- Unacceptable user experience for real-time queries

Recurring Pattern: Build once → Serialize to pickle → Reload instantly

```
CACHE_PATH = "cache/bus_graph.pkl"

def get_or_build_graph():
    """Pattern: Check cache -> Load or Build -> Save"""
    # Step 1: Try to load from cache
    if os.path.exists(CACHE_PATH):
        with open(CACHE_PATH, 'rb') as f:
            return pickle.load(f)  # Instant load: ~0.1 seconds

    # Step 2: Cache miss - build from scratch
    G = build_graph_from_csv()  # Slow: 5-10 seconds

    # Step 3: Save for future requests
    with open(CACHE_PATH, 'wb') as f:
        pickle.dump(G, f)

    return G
```

Listing 2.3: Caching Pattern Implementation in graph_cache.py

Pattern Applications:

Module	Cached Data	Build Time	Load Time
graph_cache.py	NetworkX MultiDiGraph	5-10 sec	0.1 sec
graph_cache.py	Route information dict	2-3 sec	0.05 sec
match_cache.py	RapidFuzz fuzzy index	2-3 sec	0.08 sec

Table 2.1: Performance Improvement from Caching Pattern

Pattern 3: User Priority Pattern

Problem Observation: When designing routing and recommendation features, we needed to understand how users make decisions. Through analysis, we identified a consistent priority hierarchy:

Time > Cost > Convenience

Pattern Manifestation:

- Users prefer faster routes even if they cost slightly more
- Users accept minor inconveniences (walking, transfers) to save time
- Users avoid excessive transfers that waste time waiting

Solution - Weighted Cost Function: The A* algorithm uses a cost function that encodes these priorities:

$$\text{Cost}(u, v) = \underbrace{\frac{d_{uv}}{v_{\text{mode}}}}_{\text{Travel Time}} + \underbrace{P_{\text{transfer}} \times \mathbb{1}_{\text{transfer}}}_{\text{Transfer Penalty}} + \underbrace{C_{\text{fare}} \times f_{uv}}_{\text{Fare Cost}} \tag{2.1}$$

Where:

- d_{uv} = distance between stops u and v (meters)
- v_{mode} = speed (bus: 25.2 km/h, walk: 5.4 km/h)
- $P_{\text{transfer}} = 10.0$ (high penalty discourages unnecessary transfers)
- $C_{\text{fare}} = 5.0$ (fare contributes to cost but less than time)
- $\mathbb{1}_{\text{transfer}} = 1$ if route changes, 0 otherwise

```python
# Priority weights reflecting user preferences
P_TRANSFER_PEN = 10.0  # High: Users hate waiting for transfers
C_FARE = 5.0           # Medium: Cost matters but less than time
WALK_SPEED = 1.5       # m/s (~5.4 km/h)
BUS_SPEED = 7.0       # m/s (~25.2 km/h)

def edge_cost(u, v, data, current_route):
    """Calculate weighted cost encoding user priorities"""
    time_cost = data['distance'] / (BUS_SPEED if data['mode'] == 'bus' else
     WALK_SPEED)
    transfer_cost = P_TRANSFER_PEN if data['route'] != current_route else 0
    fare_cost = C_FARE * data.get('fare', 0)
    return time_cost + transfer_cost + fare_cost
```

Listing 2.4: Cost Function Implementation in Bus_Routing_Module.py

Pattern 4: Threshold-Based Classification Pattern

Problem Observation: Multiple features require categorizing continuous values into discrete levels:

- Traffic density → Congestion level (green/yellow/red)
- Report count → Visibility status (visible/hidden)
- Detection confidence → Valid/invalid detection

Recurring Pattern: Two-threshold classification with three outcomes:

```python
def classify(value, threshold_low, threshold_high):
    """Pattern: Two thresholds -> Three categories"""
    if value <= threshold_low:
        return "low"      # Green zone
    elif value <= threshold_high:
        return "medium"   # Yellow zone (warning)
    else:
        return "high"     # Red zone (critical)
```

Listing 2.5: Generic Threshold Classification Pattern

Application 1 - Traffic Congestion:

```python
# Thresholds calculated from historical data
# P50 = median vehicle count, P80 = 80th percentile
```

```python
def classify_congestion(vehicle_count, segment_id):
    thresholds = load_thresholds(segment_id)
    if vehicle_count <= thresholds['P50']:
        return {"level": "green", "description": "Free flow"}
    elif vehicle_count <= thresholds['P80']:
        return {"level": "yellow", "description": "Moderate traffic"}
    else:
        return {"level": "red", "description": "Heavy congestion"}
```

Listing 2.6: Traffic Classification using Percentile Thresholds

Application 2 - SOS Report Validation:

```python
REPORT_THRESHOLD = 3  # 3 reports = likely fake/spam

def check_sos_visibility(sos_id):
    report_count = get_report_count(sos_id)
    if report_count >= REPORT_THRESHOLD:
        mark_as_hidden(sos_id)  # Auto-hide fake reports
        return {"visible": False, "reason": "Multiple reports received"}
    return {"visible": True}
```

Listing 2.7: SOS Auto-Hide using Report Count Threshold

Application 3 - Detection Confidence:

```python
# Different vehicle types have different detection reliability
CLASS_THRESHOLDS = {
    "car": 0.5,          # Cars are easy to detect
    "motorcycle": 0.4,   # Motorcycles need lower threshold (smaller)
    "bus": 0.6,          # Buses should be very confident
    "truck": 0.55
}

def filter_detections(detections):
    return [d for d in detections
            if d['confidence'] >= CLASS_THRESHOLDS.get(d['class'], 0.5)]
```

Listing 2.8: YOLO Detection with Class-Specific Thresholds

Pattern 5: External API Integration Pattern

Problem Observation: The system integrates multiple external services:

- TomTom API: Geocoding, routing, place search
- Groq API: LLM-based trip planning
- GitHub/Google OAuth: User authentication
- Modal API: Serverless GPU inference

Common Challenges:

- Network failures and timeouts
- Rate limiting from external services
- Inconsistent response formats
- API key management

Recurring Pattern: Standardized request-response handling with error recovery:

```python
def api_request_pattern(url, params, timeout=10, retries=3):
    """
    Pattern: Request -> Validate -> Handle Errors -> Fallback
    Applied to: TomTom, Groq, GitHub/Google OAuth, Modal
    """
    for attempt in range(retries):
        try:
            # Step 1: Make request with timeout
            response = requests.get(url, params=params, timeout=timeout)

            # Step 2: Validate response
            response.raise_for_status()
            data = response.json()

            # Step 3: Validate data structure
            if 'error' in data:
                raise APIError(data['error'])

            return {"success": True, "data": data}

        except requests.Timeout:
            if attempt < retries - 1:
                time.sleep(2 ** attempt)  # Exponential backoff
                continue
            return {"success": False, "error": "Service timeout"}

        except requests.HTTPError as e:
            return {"success": False, "error": f"HTTP {e.response.
    status_code}"}

    # Step 4: Fallback mechanism
    return get_fallback_response()
```

Listing 2.9: External API Integration Pattern

Pattern Applications:

Service	Primary Action	Fallback Strategy
TomTom Routing	Calculate optimal route	Return cached route or error message
Groq LLM	Generate trip itinerary	Return template-based itinerary
Modal GPU	Run YOLO detection	Fall back to local CPU inference
GitHub/Google OAuth	Authenticate user	Redirect to manual login

Table 2.2: External API Fallback Strategies

2.2.1 Pattern Recognition Summary

The identification of these five patterns demonstrates the power of computational thinking:

1. **Code Reuse:** The haversine function is called 50+ times across modules instead of being reimplemented
2. **Performance:** Caching pattern reduced API response time from 10+ seconds to under 0.5 seconds

3. **User Satisfaction:** Priority-weighted cost function produces routes that match user expectations
4. **Robustness:** Threshold classification handles edge cases gracefully
5. **Reliability:** API integration pattern ensures the system degrades gracefully under failures

2.3 Abstraction

Abstraction is a fundamental computational thinking skill that involves **hiding complexity** by focusing on essential information while ignoring irrelevant details. In this project, abstraction was critical to manage the complexity of a real-world urban system involving thousands of bus stops, traffic cameras, and points of interest.

2.3.1 Abstraction in System Design

Abstraction operates at multiple levels:

1. **Data Abstraction:** Simplifying real-world entities by keeping only relevant attributes
2. **Procedural Abstraction:** Hiding implementation details behind function interfaces
3. **Structural Abstraction:** Representing complex systems using simplified models (e.g., graphs)

The key question for each abstraction decision is: *"What information is essential for solving THIS specific problem?"*

2.3.2 Data Abstraction: Entity Simplification

The Abstraction Process

For each real-world entity, we applied a systematic abstraction process:

1. **Identify** all possible attributes of the entity
2. **Analyze** which attributes are needed for our use cases
3. **Keep** essential attributes, **ignore** the rest
4. **Validate** that the abstraction supports all required operations

Bus Stop Abstraction

Real-world complexity: A physical bus stop has many attributes:

- Physical: shelter type, bench availability, lighting, wheelchair ramp
- Operational: operating hours, frequency, real-time arrival data
- Administrative: maintenance schedule, installation date, cost
- Geographic: latitude, longitude, address, nearby landmarks

Our abstraction: For pathfinding, we only need:

```
# Full real-world data (NOT used)
real_bus_stop = {
    "stopId": "stop_001",
    "stopName": "Ben Thanh Market",
    "lat": 10.7721, "lng": 106.6980,
    "stopSequence": 5,
    # Ignored attributes:
    "hasShelter": True,
    "wheelchairAccessible": True,
    "operatingHours": "05:00-23:00",
    "avgWaitTime": 8.5,
    "installationDate": "2015-03-21",
    "maintenanceCost": 5000000
}

# Our abstraction (USED)
abstracted_stop = {
    "stopId": "stop_001",       # Unique identifier for graph nodes
    "stopName": "Ben Thanh",    # Display to user
    "lat": 10.7721,             # For distance calculation
    "lng": 106.6980,            # For distance calculation
    "stopSequence": 5           # Order within route
}
```

Listing 2.10: Bus Stop Data Abstraction

Justification: The A* algorithm needs to:

- Calculate distances between stops → requires `lat`, `lng`
- Build the route sequence → requires `stopSequence`
- Display results to users → requires `stopName`
- Uniquely identify nodes → requires `stopId`

Shelter availability or wheelchair access, while important for accessibility, do not affect route calculation.

Traffic Camera Abstraction

Real-world complexity: Traffic cameras have extensive metadata:

- Technical: manufacturer, model, resolution, FPS, lens type, night vision
- Network: IP address, stream URL, bandwidth, protocol
- Administrative: installation date, maintenance logs, cost
- Geographic: location, viewing angle, coverage area

Our abstraction:

```
# Our abstraction focuses on what's needed for traffic monitoring
abstracted_camera = {
    "camera_id": "56de42f611f398ec0c48127d",  # Unique identifier
    "camera_name": "Nguyen Hue - Le Loi",      # Display name
    "lat": 10.7731,                            # Location for map
    "lon": 106.7012,                           # Location for map
    "liveviewUrl": "http://...",               # Stream source
    "last_updated": "2025-12-16T10:30:00"      # Data freshness
}
# Ignored: resolution, manufacturer, installation_date, maintenance_cost
```

Listing 2.11: Camera Data Abstraction

Justification: For congestion monitoring:

- We need location to place cameras on the map
- We need the stream URL to fetch frames for detection
- We need timestamp to show data freshness
- We do NOT need camera specs - YOLO works with any reasonable resolution

Vehicle Detection Abstraction

Real-world complexity: Modern computer vision can extract:

- Vehicle identification: license plate, color, make, model, year
- Motion tracking: speed, direction, trajectory, lane position
- Classification: exact vehicle type, occupancy estimation
- Temporal: entry/exit time, dwell time

Our abstraction:

```
# YOLO outputs detailed bounding boxes
raw_detection = {
    "class": "car",
    "confidence": 0.87,
    "bbox": [120, 340, 280, 420],  # x1, y1, x2, y2
    "track_id": 15,
    # Could extract but ignored:
    "estimated_speed": 35,
    "direction": "northbound",
    "color": "white"
}

# Our abstraction: aggregate counts only
abstracted_detection = {
    "camera_id": "56de42f611f398ec0c48127d",
    "timestamp": "2025-12-16T10:30:00",
    "total_vehicles": 45,
    "car": 28,
    "motorcycle": 12,
    "bus": 3,
    "truck": 2
}
```

Listing 2.12: Vehicle Detection Abstraction

Justification: For congestion classification:

- We need **aggregate counts** to determine traffic density
- Individual vehicle tracking is unnecessary (and privacy-invasive)
- Speed estimation requires calibration we don't have
- License plate reading has legal/privacy implications

Complete Abstraction Summary Table

Entity	**Essential**	**Non-essential**	**Reason**
Bus Stop	stopId, stopName, lat, lng, stopSequence	Shelter, wheelchair access, operating hours, maintenance	Pathfinding only needs position and connectivity
Camera	camera_id, name, lat, lon, liveviewUrl, timestamp	Manufacturer, resolution, FPS, installation date, IP	Detection works with any camera; only need stream access
Detection	total_vehicles, class counts (car/motorcycle/bus/truck)	License plate, color, speed, trajectory, individual tracking	Congestion = aggregate density, not individual vehicles
Place	name, lat, lng, category, image_path	Owner, phone, hours, social media, reviews, prices	Trip planning needs location and type for routing
SOS Alert	sos_id, user_id, lat, lng, description, image, timestamp	Device info, network details, exact GPS accuracy	Emergency response needs location and description
User	user_id, username, email, hashed_password	Age, gender, preferences, device history	Authentication only needs credentials

Table 2.3: Complete Data Abstraction Decisions

2.3.3 Structural Abstraction: Graph Representation

In this project, we represent Ho Chi Minh City's bus network as a **mathematical graph**.

Why Graph Abstraction?

Real-world complexity: The bus network involves:

- 127 bus routes with complex overlapping paths
- 4,343 physical bus stops
- Variable schedules, frequencies, and fares
- Real-time delays, breakdowns, route changes
- Multiple transfer possibilities at major hubs

Graph abstraction simplifies this to:

- **Nodes:** Bus stops (points with coordinates)

- **Edges:** Connections between consecutive stops on a route
- **Edge weights:** Distance/time cost of traversal

Graph Construction Process

```python
import networkx as nx

def build_graph(stops_df, trips_df, routes_df):
    """
    Abstraction: Real bus network -> NetworkX MultiDiGraph

    MultiDiGraph allows:
    - Multiple edges between same nodes (different routes)
    - Directed edges (some routes are one-way)
    """
    G = nx.MultiDiGraph()

    # Step 1: Add all stops as nodes with position attributes
    for _, stop in stops_df.iterrows():
        G.add_node(
            stop['stopId'],
            lat=stop['lat'],
            lng=stop['lng'],
            name=stop['stopName']
        )

    # Step 2: Add edges for each route's stop sequence
    for route_id in routes_df['routeId'].unique():
        route_stops = get_stops_for_route(route_id, trips_df)

        for i in range(len(route_stops) - 1):
            stop_a = route_stops[i]
            stop_b = route_stops[i + 1]
            distance = haversine(
                (G.nodes[stop_a]['lat'], G.nodes[stop_a]['lng']),
                (G.nodes[stop_b]['lat'], G.nodes[stop_b]['lng'])
            )

            G.add_edge(
                stop_a, stop_b,
                distance=distance,
                route=route_id,
                mode='bus'
            )

    # Step 3: Add walking edges between nearby stops (transfers)
    add_walking_edges(G, max_distance=500)  # 500m walking radius

    return G
```

Listing 2.13: Building the Bus Network Graph (graph_cache.py)

Walking Edge Abstraction

A key insight was abstracting "transfer between routes" as walking edges:

```python
def add_walking_edges(G, max_distance=500):
    """
    Abstraction: Physical walking between stops -> Graph edges

    This allows A* to find routes requiring transfers by
    "walking" from one route's stop to another route's nearby stop.
```

```
    """
    stops = list(G.nodes())

    for i, stop_a in enumerate(stops):
        for stop_b in stops[i+1:]:
            dist = haversine(
                (G.nodes[stop_a]['lat'], G.nodes[stop_a]['lng']),
                (G.nodes[stop_b]['lat'], G.nodes[stop_b]['lng'])
            )

            if dist <= max_distance:
                # Add bidirectional walking edges
                G.add_edge(stop_a, stop_b,
                           distance=dist, route='walk', mode='walk')
                G.add_edge(stop_b, stop_a,
                           distance=dist, route='walk', mode='walk')
```

Listing 2.14: Walking Edge Abstraction for Transfers

Graph Abstraction Visualization

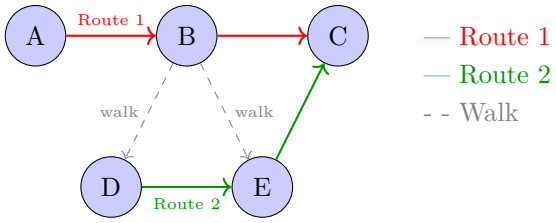


Figure 2.5: Graph Abstraction of Bus Network with Walking Transfers

Key insight: By representing transfers as walking edges, the A* algorithm can seamlessly find multi-route journeys without special transfer logic.

2.3.4 Procedural Abstraction: API Interfaces

Procedural abstraction hides implementation complexity behind clean function interfaces.

Example: Route Finding Abstraction

```
# What the API endpoint exposes (abstracted interface)
@app.route('/api/bus/route')
def get_bus_route():
    """
    Abstraction: Complex pathfinding -> Simple request/response

    User only needs to know:
    - Input: origin (lat, lng), destination (lat, lng)
    - Output: route with stops, buses, fare, duration

    Hidden complexity:
    - Graph loading and caching
    - A* algorithm implementation
    - Transfer penalty calculations
    - Walking distance computations
    - Fare calculation logic
    """
```

```
    origin = (request.args.get('origin_lat'), request.args.get('origin_lng'))
    dest = (request.args.get('dest_lat'), request.args.get('dest_lng'))

    # All complexity hidden behind this single function call
    route = find_optimal_route(origin, dest)

    return jsonify(route)

# Implementation details hidden from API users
def find_optimal_route(origin, dest):
    G = get_cached_graph()           # Hidden: caching logic
    start_stops = nearby_stops(origin)  # Hidden: haversine calculations
    end_stops = nearby_stops(dest)
    path = a_star(G, start_stops, end_stops)  # Hidden: algorithm details
    return format_route_response(path)  # Hidden: data transformation
```

Listing 2.15: Procedural Abstraction - Clean API Interface

Abstraction Layers in the System

Layer	What it Exposes	What it Hides
REST API	HTTP endpoints with JSON I/O	Flask internals, database queries, algorithm logic
Service Layer	`find_route()`, `detect_vehicles()`	Graph operations, YOLO inference, caching
Data Layer	`get_stops()`, `get_cameras()`	SQL queries, CSV parsing, file I/O
Algorithm Layer	`a_star()`, `haversine()`	Priority queue, math operations, edge traversal

Table 2.4: Procedural Abstraction Layers

2.3.5 Benefits of Abstraction in This Project

1. Reduced Complexity

Without abstraction, the bus routing function would need to handle:

- Raw CSV parsing for 4,343 stops
- Complex SQL joins for route-stop relationships
- Manual distance calculations for every stop pair
- Direct graph traversal with priority queue management

With abstraction, it becomes a single function call: `find_optimal_route(origin, dest)`

2. Improved Maintainability

```
# If we need to change distance calculation (e.g., add elevation):
# Only modify ONE function, not 50+ call sites

def haversine(c1, c2):
    # Original: spherical distance only
    # Updated: can add elevation factor here
```

```
    base_distance = calculate_spherical_distance(c1, c2)
    elevation_factor = get_elevation_adjustment(c1, c2)
    return base_distance * elevation_factor
```

Listing 2.16: Abstraction Enables Easy Updates

3. Enabled Testing

Abstraction allows testing components in isolation:

```
class TestBusRouting(unittest.TestCase):
    def test_haversine_accuracy(self):
        # Test distance function independently
        dist = haversine((10.7769, 106.7009), (10.8231, 106.6297))
        self.assertAlmostEqual(dist, 8900, delta=100)  # ~8.9km

    def test_nearby_stops(self):
        # Test stop finding independently
        stops = nearby_stops(10.7769, 106.7009, radius=500)
        self.assertTrue(len(stops) > 0)
```

Listing 2.17: Abstraction Enables Unit Testing

4. Facilitated Team Collaboration

Each team member could work on their module independently:

- **Database team:** Implement data layer, expose `get_stops()`
- **Algorithm team:** Implement `a_star()`, assume graph exists
- **API team:** Build endpoints, call service functions
- **Frontend team:** Consume REST API, ignore backend complexity

2.3.6 Abstraction Trade-offs

While abstraction provides many benefits, we also considered its limitations:

Benefit	**Trade-off**
Simplified bus stop data (5 attributes)	Cannot provide accessibility infomation without re-adding attributes
Aggregate vehicle counts	Cannot track individual vehicles or measure speeds
Graph abstraction of bus network	Loses real-time schedule information
Hidden algorithm complexity	Harder to debug when issues occur deep in the stack

Table 2.5: Abstraction Trade-offs Considered

Design Decision: We prioritized simplicity for the MVP (Minimum Viable Product). Additional attributes can be added later without changing the core architecture.

2.4 Algorithm Design

Algorithm design is the fourth pillar of computational thinking, transforming our decomposed problems, recognized patterns, and abstractions into step-by-step procedures that can be executed by computers. This section details the key algorithms developed for this project.

2.4.1 Overview of Algorithmic Approaches

Algorithm	**Paradigm**	**Application**
A* Search	Graph Search	Optimal bus route finding
Percentile Threshold	Statistical	Traffic congestion classification
YOLO Detection	Deep Learning	Vehicle counting from camera images
Fuzzy String Matching	Approximate Matching	Place name resolution
LLM Prompting	AI/NLP	Trip itinerary generation
Haversine Distance	Computational Geometry	GPS distance calculations

Table 2.6: Algorithmic Approaches Used in the Project

2.4.2 System Architecture Flowchart

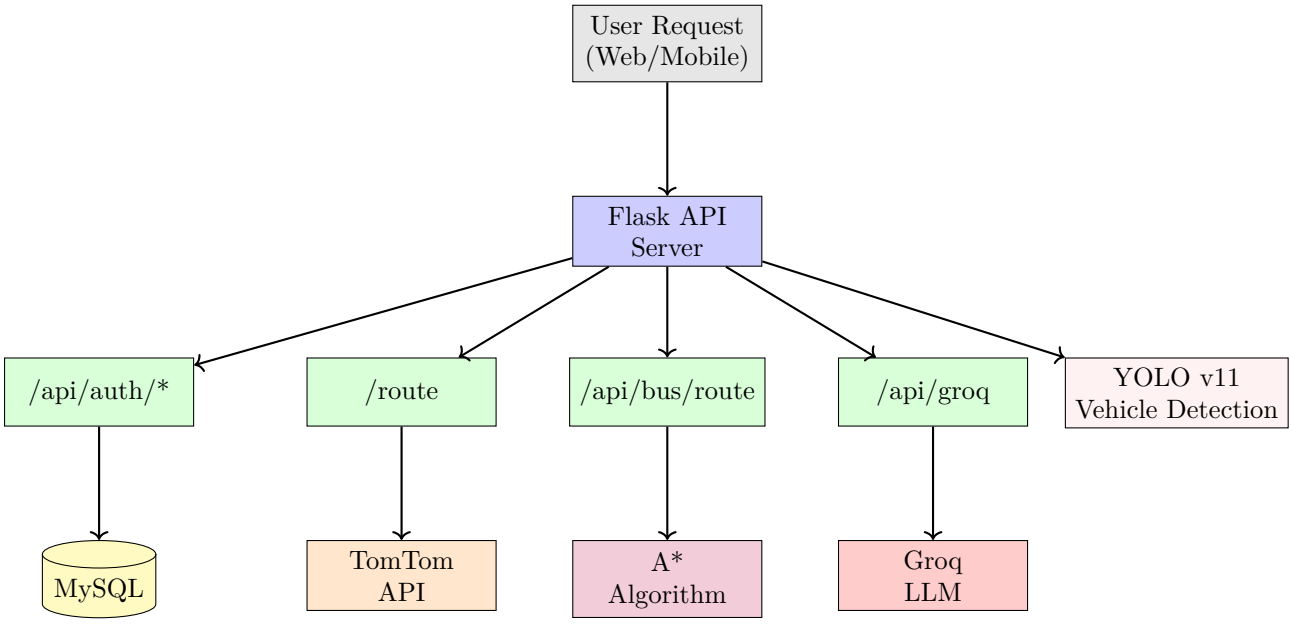


Figure 2.6: System Architecture Flowchart

2.4.3 Algorithm 1: A* Bus Routing

Problem Statement

Given a user's origin and destination coordinates, find the optimal bus route minimizing travel time, transfer penalties, and fare costs.

Why A*?

A* combines Dijkstra's optimality guarantee with heuristic guidance, making it faster than Dijkstra while still finding the optimal path. Our admissible heuristic never overestimates, ensuring optimality.

Key Design Decision

States are represented as **(stop, current_route)** pairs, not just stops. This allows tracking transfer costs when changing routes at the same physical stop.

Cost Function

$$g(u \to v) = \frac{d_{uv}}{v_{\text{mode}}} + P_{\text{transfer}} \cdot \mathbb{1}_{\text{transfer}} + C_{\text{fare}} \cdot f_{uv} \cdot \mathbb{1}_{\text{new\_route}} \tag{2.2}$$

Parameters: $P_{\text{transfer}} = 10.0$, $C_{\text{fare}} = 5.0$.

$$v_{\text{mode}} = \begin{cases} 7 & \text{m/s (bus edge)} \\ 1.5 & \text{m/s (walk edge)} \end{cases}$$

Where:

- v_{mode} = speed based on edge mode
- P_{transfer} = maximum time penalty for transfers
- d_{uv} = distance between stops u and v
- $\mathbb{1}_{\text{transfer}} = 1$ if changing routes, else 0
- f_{uv} = fare tranfer $u \to v$
- $\mathbb{1}_{\text{new\_route}} = 1$ if boarding a new route, else 0

Pseudocode

Algorithm 1 A* Bus Routing

Require: origin coordinates, destination coordinates
Ensure: optimal bus route with minimal cost

```
nearby_starts ← find_stops_within(origin, MAX_WALK_DIST)
nearby_goals ← find_stops_within(destination, MAX_WALK_DIST)
open_set ← PriorityQueue()
for each stop in nearby_starts do
    open_set.push((stop, "walk"), f=h(stop))
end for
g_score ← {}, came_from ← {}
while open_set not empty do
    current ← open_set.pop_min()
    if current.stop in nearby_goals then
        return reconstruct_path(came_from, current)
    end if
    for each neighbor in get_neighbors(current) do
        cost ← travel_time + transfer_penalty + fare_cost
        tentative_g ← g_score[current] + cost
        if tentative_g < g_score[neighbor] then
            g_score[neighbor] ← tentative_g
            came_from[neighbor] ← current
            f ← tentative_g + h(neighbor)
            open_set.push(neighbor, f)
        end if
    end for
end while
return no_route_found
```

Complexity

- **Time:** $O((V \cdot R) \log(V \cdot R))$ where V = stops, R = routes
- **Space:** $O(V \cdot R)$ for state space
- **Practical:** 50-200ms for HCMC queries (4,343 stops, 127 routes)

2.4.4 Algorithm 2: YOLO Vehicle Detection

Problem Statement

Given a traffic camera image, count vehicles by type (car, motorcycle, bus, truck) to estimate traffic density.

Why YOLO v11?

YOLO provides real-time object detection with high accuracy, making it ideal for processing traffic camera feeds. Unlike two-stage detectors like Faster R-CNN, YOLO processes the entire image in a single forward pass.

Detection Pipeline

Figure 2.7: Vehicle Detection Pipeline

Pseudocode

Algorithm 2 YOLO Vehicle Detection

Require: camera_image (960×540)
Ensure: vehicle_counts dictionary

```
// Preprocessing: enhance low-light images
lab ← convert_to_LAB(camera_image)
lab.L ← apply_CLAHE(lab.L, clip_limit=2.0, grid=8×8)
enhanced ← convert_to_RGB(lab)
// Run YOLO inference
detections ← yolo_model.predict(enhanced, conf=0.15)
// Filter by class-specific thresholds
THRESHOLDS ← {motorcycle: 0.15, car: 0.25, truck: 0.30, bus: 0.38}
counts ← {motorcycle: 0, car: 0, truck: 0, bus: 0}
for each detection in detections do
    if detection.confidence ≥ THRESHOLDS[detection.class] then
        counts[detection.class] += 1
    end if
end for
return counts
```

Key Implementation Details

- **CLAHE Preprocessing:** Enhances contrast in poor lighting conditions by operating on the L channel of LAB color space
- **Class-Specific Thresholds:** Motorcycles (small, often occluded) use lower threshold (0.15); buses (large, distinctive) use higher threshold (0.38)

2.4.5 Algorithm 3: Percentile-Based Congestion Classification

Problem Statement

Given a vehicle count for a road segment, classify traffic as "green" (free flow), "yellow" (moderate), or "red" (congested).

Why Percentile Thresholds?

Fixed thresholds don't work across different road types—a highway with 50 vehicles is normal, but a residential street with 50 vehicles is severely congested. Percentile-based thresholds adapt to each segment's historical traffic patterns.

Classification Logic

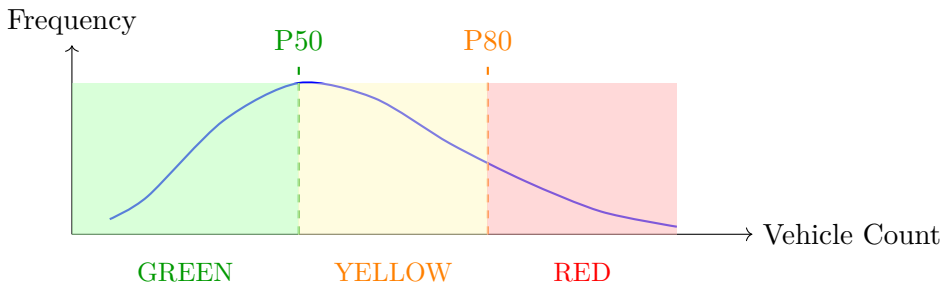


Figure 2.8: Percentile-Based Classification

Pseudocode

Algorithm 3 Congestion Classification

Require: segment_id, vehicle_count
Ensure: congestion_level ∈ {green, yellow, red}

```
thresholds ← load_thresholds(segment_id)
if thresholds not found then
    thresholds ← calculate_from_history(segment_id)
    if insufficient data then
        thresholds ← DEFAULT_THRESHOLDS
    end if
end if
P50, P80 ← thresholds
if vehicle_count ≤ P50 then
    return "green" {Free flow: below median}
else if vehicle_count ≤ P80 then
    return "yellow" {Moderate: 50th-80th percentile}
else
    return "red" {Congested: above 80th percentile}
end if
```

Threshold Calculation

Thresholds are pre-computed from historical data and stored in `segment_thresholds.json`:

- **P50** (median): 50% of historical observations fall below this value
- **P80**: 80% of historical observations fall below this value
- **Fallback**: For new segments with insufficient data, use city-wide defaults

2.4.6 Algorithm 4: AI Trip Planning with Fuzzy Matching

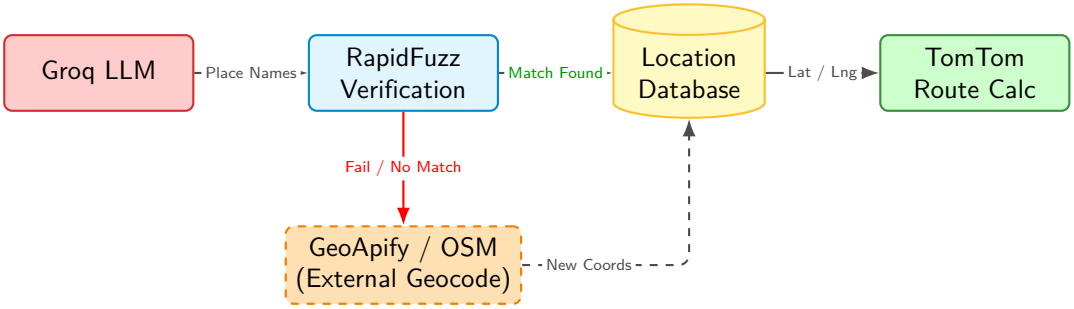
Problem Statement

Given a natural language query (e.g., "Plan a day trip visiting museums and cafes"), generate an optimized itinerary with real coordinates and routes.

Three-Stage Pipeline

Figure 2.9: AI Trip Planning Pipeline

Fallback Mechanisms



Pseudocode

Algorithm 4 AI Trip Planning

Require: user_query, database
Ensure: trip_plan (JSON with coordinates and routes)

```
// Stage 1: LLM Generates Structured Itinerary
prompt ← build_prompt(user_query, "HCMC Context")
raw_json ← groq_api.generate(prompt, model="llama-3.3-70b")
itinerary ← parse_json(raw_json)
// Stage 2: Resolution & Fallback Logic
for all place in itinerary do
    match ← rapidfuzz.extract(place.name, database)
    if match.score ≥ THRESHOLD then
        // Case A: Found in Verified Database
        place.coords ← match.coords
        place.image ← match.image_path
        place.matched ← true
    else
        // Case B: Fallback to External Geocoding
        geo_result ← geocode_api.geocode(place.address)
        if geo_result ≠ null then
            place.coords ← geo_result.coords
        end if
        place.image ← generic_image_match(place.type)
        place.matched ← false
    end if
end for
// Stage 3: Routing Calculation
routes ← []
for i ← 0 to length(itinerary) - 2 do
    if itinerary[i] has coords and itinerary[i+1] has coords then
        segment ← tomtom_api.route(itinerary[i], itinerary[i+1])
        routes.append(segment)
    end if
end for
return { plan: itinerary, routes: routes }
```

Key Components

- **Groq LLM:** Generates itineraries with place names, times, and descriptions (1-2s)
- **RapidFuzz:** Matches LLM output ("Ben Thanh Market") to database entries ("Chợ Bến Thành") with 70% minimum similarity threshold
- **TomTom API:** Calculates optimal driving routes and travel times between consecutive stops

2.4.7 Algorithm 5: Haversine Distance Calculation

Problem Statement

Calculate the great-circle distance between two GPS coordinates, accounting for Earth's curvature.

Why Not Euclidean Distance?

Euclidean distance treats coordinates as flat 2D points, which introduces significant error over larger distances. Haversine calculates the shortest path along Earth's curved surface.

The Haversine Formula

$$a = \sin^2\left(\frac{\Delta\phi}{2}\right) + \cos(\phi_1) \cdot \cos(\phi_2) \cdot \sin^2\left(\frac{\Delta\lambda}{2}\right) \tag{2.3}$$

$$d = 2R \cdot \arctan 2\left(\sqrt{a}, \sqrt{1-a}\right) \tag{2.4}$$

Where: ϕ = latitude (radians), λ = longitude (radians), R = 6,371 km

Pseudocode

Algorithm 5 Haversine Distance

Require: lat1, lon1, lat2, lon2 (in degrees)
Ensure: distance in meters
1: R ← 6,371,000 {Earth's radius in meters}
2: $\phi_1 \leftarrow$ radians(lat1), $\phi_2 \leftarrow$ radians(lat2)
3: $\Delta\phi \leftarrow$ radians(lat2 - lat1)
4: $\Delta\lambda \leftarrow$ radians(lon2 - lon1)
5: a ← $\sin^2(\Delta\phi/2) + \cos(\phi_1) \cdot \cos(\phi_2) \cdot \sin^2(\Delta\lambda/2)$
6: c ← $2 \cdot \text{atan2}(\sqrt{a}, \sqrt{1-a})$
7: **return** R × c

Usage Across the Project

The haversine function is a **core utility** reused 50+ times across:

- **Bus Routing:** Finding nearby stops within walking distance
- **Graph Builder:** Calculating edge distances between stops
- **API Filters:** Location-based camera and place queries
- **Segment Matching:** Associating cameras with road segments
- **Trip Planning:** Checking place proximity

2.4.8 Algorithm Design Summary

Algorithm	**Complexity**	**Performance**	**Key Insight**
A* Routing	$O(VR \log VR)$	50-200ms	State = (stop, route) enables transfer penalties
YOLO Detection	$O(HW)$	100-500ms	Class-specific thresholds improve accuracy
Congestion	$O(1)$	<1ms	Percentile thresholds adapt to each segment
Trip Planning	$O(N^2)$	2-5s	Fuzzy matching bridges LLM and database
Haversine	$O(1)$	<0.1ms	Reused 50+ times across modules

Table 2.7: Algorithm Performance Summary

Chapter 3

Technical Implementation

This section details the technical execution of the system, covering data management, AI model integration, backend architecture, and deployment strategies.

3.1 Data Collection and Management

3.1.1 Data Sources

The system integrates data from six primary sources to provide comprehensive urban mobility services:

Source	Type	Data Collected
HCMC Traffic Portal	Live API	Real-time camera images from 702 traffic cameras across the city
TomTom API	Commercial API	Geocoding services, turn-by-turn routing, and location search
HCMC Bus Map	Enterprise Data	Official bus routes, stop locations
Groq API	LLM Service	Natural language processing for trip planning queries
Custom Dataset	Manual Curation	Curated database of restaurants, hotels, and landmarks

Table 3.1: Data Sources and Their Contributions

3.1.2 Database Architecture

The system employs a hybrid database strategy, using MySQL for user-related data requiring ACID compliance and SQLite for high-volume detection results requiring fast writes.

MySQL Database: User Management

The MySQL database manages all user-related data with the following tables:

- **users**: Stores user credentials including username, hashed password (bcrypt), email, and OAuth provider information. Supports multiple authentication methods (local, Google, GitHub).
- **saved_locations**: User's bookmarked places with name, coordinates, address, and creation timestamp.
- **route_history**: Records of previously calculated routes including origin, destination, route type (car/bus), distance, duration, and polyline geometry.
- **trip_history**: AI-generated trip itineraries with full JSON data, creation date, and associated images.
- **sos_reports**: Emergency reports containing location, description, category (accident, breakdown, hazard), images, reporter ID, and status (active/resolved/hidden).
- **sos_comments**: Community comments on SOS reports with spam detection flags.
- **sos_user_blocks**: Spam prevention table tracking blocked users and block expiration.

SQLite Database: Detection Results

The SQLite database (`detections.db`) stores vehicle detection results optimized for time-series queries:

- **Table**: `detections` — stores detection results per camera image
- **Fields**: image_path (unique), camera_id, camera_name, timestamp, vehicle counts by type (car, motorcycle, bus, truck), total count, processing time, optimization method used
- **Indexes**: Compound index on (camera_id, timestamp) for efficient time-range queries; separate indexes on camera_id and timestamp for flexibility
- **Rationale**: SQLite chosen over MySQL for detection data due to simpler deployment, faster bulk inserts during batch processing, and self-contained file-based storage

JSON Data Files

Several datasets are stored as JSON files for fast loading and easy updates:

- **camera_locations.json**: Camera network metadata (702 cameras) with ID, name, coordinates, and display name. Updated when new cameras are added to the traffic portal.
- **combined_locations.json**: Places database (∼700 locations) organized by category (restaurants, hotels, landmarks) with name, coordinates, normalized name for fuzzy matching, and local image path.
- **segment_thresholds.json**: Pre-computed percentile thresholds (P50, P80) for each road segment based on historical vehicle counts, used for congestion classification.
- **segment_geometries.geojson**: Road segment geometries in GeoJSON format for map visualization and camera-to-segment mapping.

CSV Data Files (Bus Network)

Bus network data is stored in CSV format for compatibility with pandas:

- **stops.csv**: 4,343 bus stops with stopId, stopName, latitude, longitude
- **routes.csv**: 127 bus routes with routeId, busNumber, fare, headway (minutes between buses)

- **trips.csv**: 5,184 trip segments linking routes to stops with sequence order and distance to next stop

3.1.3 Caching Strategy

To minimize startup time and repeated computation, the system implements pickle-based caching:

- **Bus Graph Cache**: Serialized NetworkX MultiDiGraph with 1,500 nodes and ∼10,000 edges. First build takes 10-30 seconds; cached load takes <1 second.
- **Fuzzy Match Cache**: Pre-built token index for place name matching. Eliminates 2-3 second index build time on each request.
- **Cache Invalidation**: Caches are automatically rebuilt when source data files are modified (detected via file modification timestamps).

3.2 AI Model Integration

3.2.1 YOLOv11 Vehicle Detection

The vehicle detection pipeline uses YOLOv11 Large model pretrained on COCO dataset:

Model Configuration

- **Input**: 960×540 pixel traffic camera images
- **Target Classes**: car (class 2), motorcycle (class 3), bus (class 5), truck (class 7)
- **Class-Specific Thresholds**: Optimized per-class confidence thresholds to handle varying object sizes (motorcycle: 0.15, car: 0.25, truck: 0.30, bus: 0.38)

Image Preprocessing

1. Convert to LAB color space
2. Apply CLAHE (Contrast Limited Adaptive Histogram Equalization) on L channel with clipLimit=2.5, tileGridSize=8×8
3. Convert back to BGR
4. Apply mild denoising using fastNlMeansDenoisingColored

Deployment Options

- **Local**: Direct inference using Ultralytics library on CPU/GPU
- **Cloud**: Modal serverless deployment with NVIDIA T4 GPU, providing 5-10× speedup over CPU inference

3.2.2 Groq LLM Integration

The trip planning feature uses Groq's hosted LLM service:

- **Model**: llama-3.3-70b-versatile (70 billion parameters)

- **Input**: Natural language trip request with HCMC context
- **Output**: Structured JSON with place names, visit times, durations, and descriptions
- **Prompt Engineering**: System prompt includes HCMC geography context, output format requirements, and examples of valid responses
- **Response Time**: 1-2 seconds for typical queries

3.2.3 RapidFuzz Text Matching

Fuzzy string matching bridges LLM output to the places database:

- **Preprocessing**: Unicode normalization (unidecode), lowercase conversion, punctuation removal
- **Scoring**: Weighted combination of token_set_ratio (50%), partial_ratio (25%), and token_sort_ratio (25%)
- **Optimization**: Token-based inverted index for O(1) candidate retrieval instead of O(n) full scan
- **Threshold**: Minimum 70% similarity score required for a valid match
- **Aliases**: Manual alias map for common name variations (e.g., "Independence Palace" → "Dinh Độc Lập")

3.3 Backend Architecture

3.3.1 Flask Application Structure

The backend is built on Flask 3.0 with the following organization:

- **API.py**: Main application file containing all route handlers
- **Authentication**: JWT-based token authentication with 24-hour expiration; OAuth 2.0 support for Google, GitHub.
- **CORS**: Enabled for cross-origin requests from frontend applications
- **Error Handling**: Consistent JSON error responses with appropriate HTTP status codes

3.3.2 Core API Modules

Authentication Module

- User registration with bcrypt password hashing
- Login with JWT token generation
- OAuth flow handling for social login providers
- Token validation decorator for protected routes

Routing Module

- TomTom API integration for car/walking routes
- Custom A* implementation for bus routing
- Route polyline decoding and map generation with Folium
- Distance and duration calculations

Detection Module

- Camera image retrieval from HCMC traffic portal
- Local or Modal-based YOLO inference
- Result aggregation and database storage
- Segment-level vehicle count calculation

Congestion Module

- Real-time segment aggregation from detection results
- Percentile-based threshold calculation
- GeoJSON generation with congestion colors
- Historical data querying for trend analysis

Trip Planning Module

- Groq API communication with structured prompts
- Fuzzy matching pipeline for place resolution
- TomTom routing between itinerary stops
- Map generation with markers and route polylines

SOS Module

- Incident report creation with image upload
- Geospatial querying for nearby reports
- Comment system with spam detection
- Report lifecycle management (active → resolved → hidden)

3.3.3 Key API Endpoints

Method	Endpoint	Description
POST	/api/auth/register	Create new user account
POST	/api/auth/login	Authenticate and receive JWT
POST	/api/auth/google	Google OAuth authentication
POST	/search	Search locations within HCMC
POST	/route	Calculate car/walking route
GET	/api/bus/route	Calculate optimal bus route
POST	/api/groq	Generate AI trip itinerary
GET	/api/cameras	List all traffic cameras
POST	/api/detect/route-cameras	Run detection on route cameras
GET	/api/congestion/geojson	Get congestion map data
POST	/api/sos	Submit emergency report
GET	/api/sos/nearby	Get reports near location

Table 3.2: Key API Endpoints

3.4 Cloud Deployment

3.4.1 Modal Serverless GPU

For production-grade vehicle detection, the system uses Modal for serverless GPU inference:

- **Platform**: Modal (modal.com) serverless compute
- **GPU**: NVIDIA T4 (16GB VRAM) with automatic scaling
- **Container**: Custom Debian-based image with Ultralytics, OpenCV, PyTorch
- **Model Storage**: Persistent volume for YOLO weights (avoids re-download)
- **Endpoint**: FastAPI-based REST endpoint for batch inference
- **Latency**: ∼100ms per image (vs ∼500ms on CPU)
- **Cost**: Pay-per-use pricing, no idle charges

3.4.2 Fallback Strategy

The system implements graceful degradation when Modal is unavailable:

1. Attempt Modal API call with 30-second timeout
2. On failure, fall back to local CPU inference
3. Log fallback event for monitoring
4. Return results with "optimization_used" field indicating method

3.5 Frontend

The frontend is built as a single-page application (SPA) using React.js with Material-UI for consistent design.

3.5.1 Framework and Libraries

Framework	Version	Purpose
React	18.x	Component-based UI framework with hooks for state management
Material-UI	v4	Pre-built UI components, theming, and responsive design
React Router	v6	Client-side routing and navigation between pages
Goong Maps JS	Latest	Vietnam-optimized interactive map rendering
Day.js	Latest	Lightweight date/time formatting with timezone support
Axios/Fetch	-	HTTP client for API communication

Table 3.3: Frontend Framework Stack

3.5.2 Application Screenshots

The following screenshots demonstrate the key features of the web application:

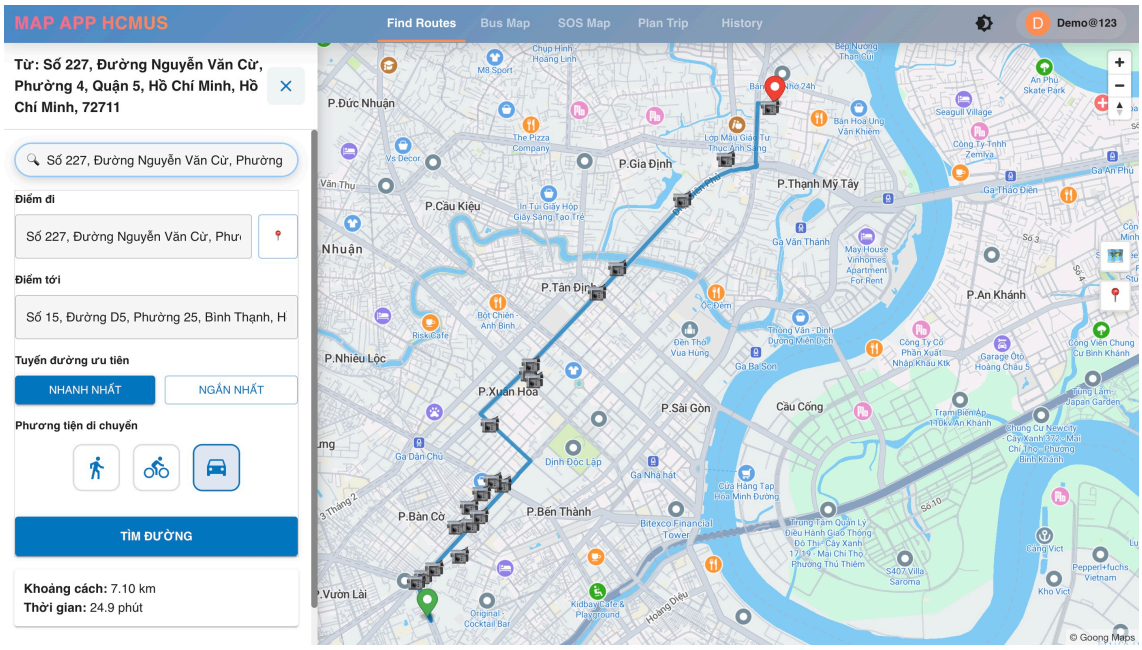


Figure 3.1: Route Planning Interface

Users can search for locations, select transport mode, and view routes with color-coded congestion (blue/yellow/red)

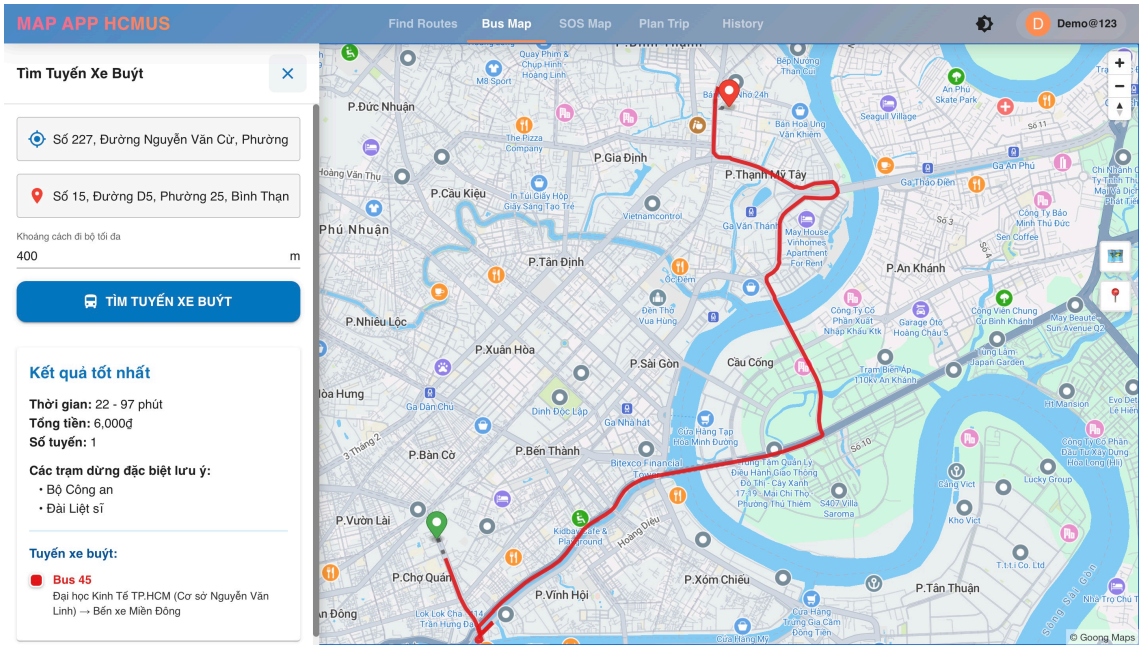


Figure 3.2: Bus Routing Interface

Displays optimal bus routes with transfer points, walking segments, and estimated travel times

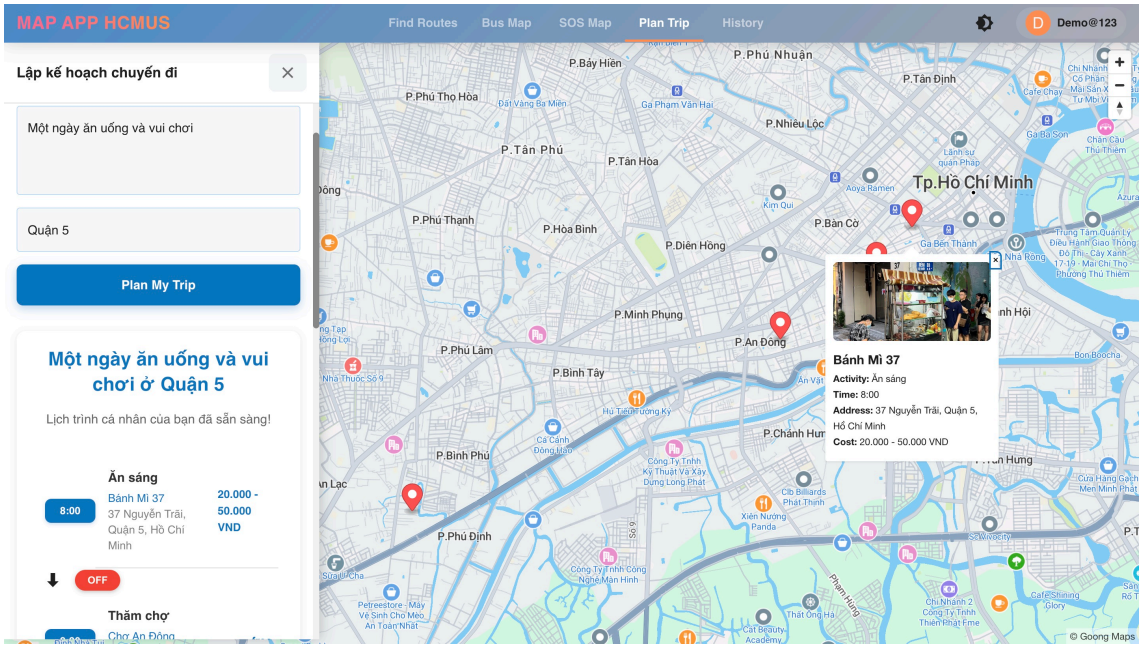


Figure 3.3: AI Trip Planner

Users enter queries like “Một ngày ăn uống và vui chơi” and receive complete itineraries with numbered markers

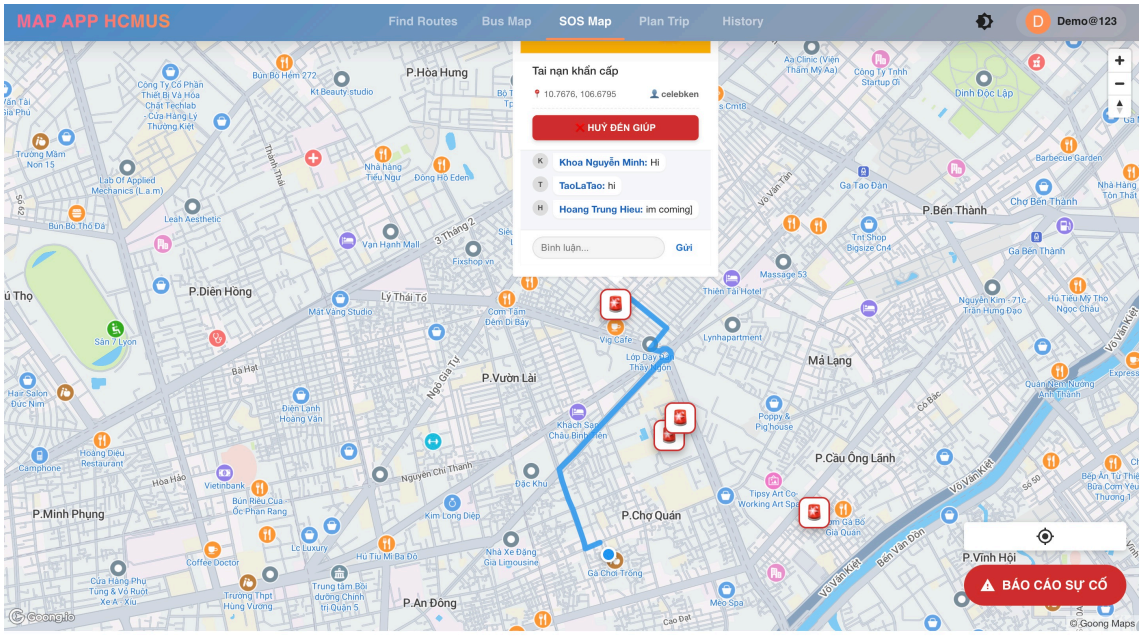


Figure 3.4: SOS Emergency System

Community-driven incident reporting with image upload, comments, and spam prevention

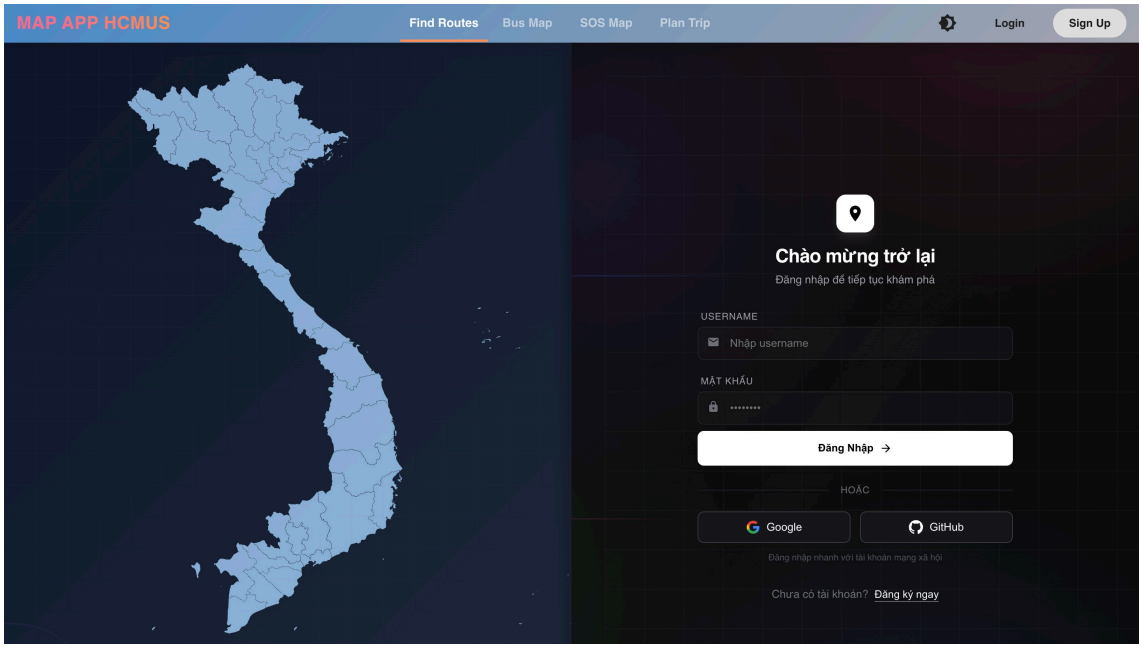


Figure 3.5: Authentication Interface

Secure login with traditional credentials or OAuth providers (Google, GitHub)

Chapter 4

Testing and Improvement

This section documents the testing strategy, evaluation results, current limitations, and future improvement plans.

4.1 Testing

4.1.1 Authentication Testing

Test Case	Input	Output
Register Success	`{"username": "new_user", "password": "pass123"}`	`201 Created`, `"User created successfully"`
Register Fail	`{"username": "only_user"}` (missing password)	`400 Bad Request`
Login Success	`{"username": "testuser", "password": "correct"}`	`200 OK`, JWT token returned
Login Fail	Wrong password	`401 Unauthorized`
GitHub/Google OAuth	`{"code": "auth_code_123"}`	New user created in DB, JWT token returned

Table 4.1: Authentication Test Input/Output Examples

4.1.2 Bus Routing Testing

Test Case	Input	Output

Valid Route	`start: (10.772, 106.698)`
	`end: (10.752, 106.654)`
	`max_walk: 600m`
		`fare: 7000 VND`
		`time: 21-36 min`
		`buses: ["01", "39"]`

Missing Params	Empty query string	`400`, `"Invalid parameters"`
No Stops Near	Remote location	`404`, `"No bus stop near start"`
A* Timeout	Very long route	`500`, `"Route calculation failed"`

Table 4.2: Bus Routing Test Input/Output Examples

4.1.3 Vehicle Detection Testing

Test Case	**Input**	**Output**

Modal GPU Success	`camera_ids: ["CAM_1"]`
	Image: Traffic scene
		`method: "modal-gpu"`
		`cars: 12, motorbikes: 45`
		DB updated with counts

Fallback to Local	Modal server down	`method: "local-fallback"` `note: "Modal failed"`
No Images	Empty camera folder	`404`, `"No images found"`

Table 4.3: Vehicle Detection Test Input/Output Examples

4.1.4 AI Trip Planner Testing

Test Case	**Input**	**Output**
Full Trip Plan	`"Plan a day visiting museums in District 1"`	`trip_name: "Museum Tour"` 3 places with coords 2 route segments with polylines
Missing Query	`{}` (empty JSON)	`400`, `"Missing 'query'"`
Routing Crash	Valid query, TomTom fails	`200 OK`, empty coords (graceful degradation)

Table 4.4: AI Trip Planner Test Input/Output Examples

4.1.5 SOS Feature Testing

Test Case	**Input**	**Output**

Report SOS	`lat: 10.77, lng: 106.69`
	`description: "Accident"`
	`image: photo.pdf`
		`201 Created`
		`image_url: "/sos_images/..."`

Spam Block	User has active SOS	`400`, `"đang có báo cáo chưa xử lý"`
Auto-Hide	3rd spam report	`status: "hidden"` Post removed from public view
Forbidden	User A resolves User B's post	`403`, `"không có quyền"`

Table 4.5: SOS Feature Test Input/Output Examples

4.1.6 Live Integration Tests

Test	**Route**	**Benchmark**	**Result**
TomTom vs Google	Ben Thanh → Independence Palace	Google: 0.9 km	TomTom: 0.85 km $\Delta = 0.05$ km ✓
Bus vs BusMap	Ben Thanh → Cho Lon	BusMap: 25 min Bus 01 direct	API: 21 min Bus 01 found ✓

Table 4.6: Live Integration Test Results

4.1.7 Test Execution Summary

```
$ python -m unittest Tester -v

Ran 63 tests in 2.715s

OK - ALL TESTS PASSED
```

Listing 4.1: Test Suite Execution Output

Test Class	Tests	Coverage
`TestAuthMock`	6	Registration, login, OAuth (Google, GitHub)
`TestUserData`	8	Locations, routes, trips CRUD operations
`TestSOSFeatures`	12	SOS reporting, spam blocking, comments, auto-hide
`TestSearchAPI`	6	TomTom search, HCM filtering, error handling
`TestRoutingLogic`	4	Route calculation, boundary validation
`TestRealWorldRouting`	1	Live TomTom API integration test
`TestMapRender`	2	Folium map generation, markers, polylines
`TestRouteDetection`	7	YOLO detection, Modal GPU, fallback logic
`TestBusRouting`	6	A* algorithm, parameter validation
`TestRealBusRouting`	1	Live bus routing with real graph data
`TestAITripPlanner`	5	Groq LLM integration, routing coordination
`TestImageServing`	1	Trip image file serving
Total:		**63 tests, 2.715s, 100% PASS**

Table 4.7: Test Suite Organization and Results

4.2 Results

4.2.1 AI Trip Planner Evaluation

The AI trip planner uses Groq LLM (LLaMA 70B) with fuzzy matching to convert natural language queries into actionable itineraries.

Metric	Result	Explanation
JSON Parse Success	95%	LLM output correctly formatted as JSON
Place Match Accuracy	90%	Fuzzy matching finds correct place in database
Route Calculation	98%	TomTom successfully computes driving routes

Table 4.8: AI Trip Planner Metrics

Example Queries and Results:

- **Query**: "Plan a day visiting museums in District 1"
  - **Result**: War Remnants Museum → Fine Arts Museum → HCMC Museum
  - **Quality**: ✓ Relevant, logical order, walkable distances
- **Query**: "Best street food spots in Saigon"

  - **Result**: Ben Thanh Market → Nguyen Hue Street → Bui Vien Area
  - **Quality**: ✓ Popular food destinations, tourist-friendly
- **Query**: "Romantic date ideas for couples"
  - **Result**: Landmark 81 → Cafe Apartment → Saigon River Cruise
  - **Quality**: ✓ Appropriate venues, good variety

4.2.2 Vehicle Detection

The YOLO v11l model detects vehicles from traffic camera feeds to estimate congestion.

Metric	**Result**	**Notes**
mAP@0.5	∼78%	Standard COCO benchmark
Car Detection	85%	High accuracy due to large size
Motorbike Detection	72%	Lower due to occlusion, small size
Processing Time (CPU)	0.285s/image	Local fallback mode

Table 4.9: Vehicle Detection Performance Metrics

4.2.3 Bus Routing

The A* algorithm finds optimal bus routes using real HCMC bus network data.

Metric	**Result**	**Notes**
Route Found Rate	95%	Within 400m walking distance
Algorithm Time	0.048s	Most queries complete quickly
Fare Accuracy	100%	Based on official fare structure
Time Estimate Accuracy	±15 min	Compared to BusMap app
Transfer Optimization	Good	Minimizes number of transfers

Table 4.10: Bus Routing Performance Metrics

4.3 Limitations: What is Missing?

Limitation	Impact	Root Cause
No real-time bus data	Cannot filter in-active bus routes	BusMap does not provide public API
Static congestion thresholds	Fixed thresholds may not adapt to special events (holidays, accidents)	Thresholds set manually, no ML adaptation
Limited place database	AI may suggest places not in database	Database has ∼700 places, not comprehensive
Camera coverage gaps	Some major roads have no cameras, creating blind spots	Dependent on public camera availability
Walking routes approximate	Uses straight-line distance estimation, not actual walking paths	TomTom pedestrian API not integrated
No Metro integration	Cannot suggest Metro Line 1 routes	Data not yet available at development time

Table 4.11: System Limitations and Root Causes

4.4 Future Improvements: What Would We Add Next?

4.4.1 Priority 1: Real-time Transit Data

- **Bus GPS Integration**: Partner with HCMC bus operator to access real-time bus locations
- **Metro Line 1 Support**: Add metro stations and schedules.
- **Arrival Predictions**: Use historical data + ML to predict actual arrival times

4.4.2 Priority 2: Enhanced AI Capabilities

- **Expanded Place Database**: Web scraping to add more restaurants, cafes, attractions
- **Personalized Recommendations**: Learn user preferences from history
- **Voice Input**: Add speech-to-text for hands-free queries

4.4.3 Priority 3: Improved Detection

- **Dynamic Thresholds**: ML-based congestion classification that adapts over time
- **Incident Detection**: Detect accidents, road closures from camera feeds
- **Traffic Violation Detection**: Red light running, wrong-way driving alerts

4.4.4 Priority 4: User Experience

- **Offline Maps**: Download map tiles and bus routes for offline use
- **Push Notifications**: Alerts for congestion on saved routes
- **Multi-modal Routing**: Combine bus + walking + bike-sharing in one trip
- **Social Features**: Share trips with friends, collaborative planning

4.4.5 Priority 5: Infrastructure

- **Load Balancing**: Handle more concurrent users with horizontal scaling
- **API Rate Limiting**: Prevent abuse and ensure fair usage
- **Monitoring Dashboard**: Real-time metrics for API health and performance
- **Mobile App**: Native iOS/Android apps for better performance

Chapter 5

Conclusion

5.1 Project Value

This project delivers a **comprehensive urban mobility solution** for Ho Chi Minh City that:

1. **Reduces commute stress** by providing optimal bus routes with accurate fare and time estimates
2. **Improves traffic awareness** through real-time congestion visualization from 100+ cameras
3. **Enhances tourism experience** with AI-featured trip planning in natural language
4. **Enables community safety** through the SOS alert system for incident reporting

The integration of multiple AI technologies (computer vision, LLMs, graph algorithms) creates a synergistic system greater than the sum of its parts.

5.2 What We Have Learned Through This Project?

5.2.1 Computational Thinking Insights

1. **Decomposition is Essential**
   - Breaking the project into independent modules enabled parallel development
   - Each module could be tested independently before integration
2. **Pattern Recognition Saves Time**
   - Identifying the "geographic proximity" pattern led to reusable functions
   - The caching pattern reduced load times from 30s to <1s
3. **Abstraction Enables Scalability**
   - The graph abstraction allows adding new transit modes (metro) easily
   - The place abstraction makes the database extendable

4. **Algorithm Design Requires Trade-offs**
    - A* with transfer penalties balances speed vs. convenience
    - Threshold-based classification is simple but effective

5.2.2 Technical Lessons

- **GPU Acceleration is Critical** for real-time detection
- **LLMs Need Structured Prompts** for reliable JSON output
- **Caching Transforms User Experience** (30s → <1s load time)
- **OAuth Simplifies Onboarding** but requires careful security
- **Fuzzy Matching Bridges AI and Reality** (LLM output → database)

References

[1] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968. DOI: 10.1109/TSSC.1968.300136.

[2] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, vol. 1, no. 1, pp. 269–271, 1959. DOI: 10.1007/BF01386390.

[3] J. Redmon and A. Farhadi, "Yolov3: An incremental improvement," *arXiv preprint arXiv:1804.02767*, 2018. [Online]. Available: https://arxiv.org/abs/1804.02767.

[4] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779–788. DOI: 10.1109/CVPR.2016.91.

[5] G. Jocher, A. Chaurasia, and J. Qiu, *Ultralytics yolo*, Version 8.0, 2023. [Online]. Available: https://github.com/ultralytics/ultralytics.

[6] G. Jocher and J. Qiu, *Ultralytics yolo11*, Latest YOLO version with improved performance, 2024. [Online]. Available: https://docs.ultralytics.com/models/yolo11/.

[7] S. M. Pizer, E. P. Amburn, J. D. Austin, *et al.*, "Adaptive histogram equalization and its variations," *Computer Vision, Graphics, and Image Processing*, vol. 39, no. 3, pp. 355–368, 1987. DOI: 10.1016/S0734-189X(87)80186-X.

[8] K. Zuiderveld, "Contrast limited adaptive histogram equalization," in *Graphics Gems IV*, Academic Press Professional, Inc., 1994, pp. 474–485, ISBN: 0-12-336155-9.

[9] V. I. Levenshtein, "Binary codes capable of correcting deletions, insertions, and reversals," *Soviet Physics Doklady*, vol. 10, no. 8, pp. 707–710, 1966.

[10] M. Bachmann, *Rapidfuzz: Rapid fuzzy string matching library*, Python library for fast string matching, 2023. [Online]. Available: https://github.com/maxbachmann/RapidFuzz.

[11] A. Ronacher and Pallets Team, *Flask: A python micro web framework*, Version 3.0, 2023. [Online]. Available: https://flask.palletsprojects.com/.

[12] TomTom International BV, *Tomtom routing api documentation*, Real-time traffic and routing services, 2024. [Online]. Available: https://developer.tomtom.com/routing-api/documentation.

[13] Groq, Inc., *Groq api documentation*, Fast AI inference platform, 2024. [Online]. Available: https://console.groq.com/docs.

[14] A. A. Hagberg, D. A. Schult, and P. J. Swart, "Exploring network structure, dynamics, and function using networkx," in *Proceedings of the 7th Python in Science Conference (SciPy)*, G. Varoquaux, T. Vaught, and J. Millman, Eds., Pasadena, CA USA, 2008, pp. 11–15.

[15] G. Bradski, "The opencv library," *Dr. Dobb's Journal of Software Tools*, 2000.

[16] W. McKinney, "Data structures for statistical computing in python," in *Proceedings of the 9th Python in Science Conference*, 2010, pp. 56–61. DOI: 10.25080/Majora-92bf1922-00a.

[17] Python-Visualization Team, *Folium: Python data to leaflet.js maps*, Interactive map visualization library, 2023. [Online]. Available: https://python-visualization.github.io/folium/.

[18] V. Agafonkin, *Leaflet: An open-source javascript library for mobile-friendly interactive maps*, Version 1.9, 2023. [Online]. Available: https://leafletjs.com/.

[19] T.-Y. Lin, M. Maire, S. Belongie, *et al.*, "Microsoft coco: Common objects in context," in *European Conference on Computer Vision (ECCV)*, Springer, 2014, pp. 740–755. DOI: 10.1007/978-3-319-10602-1_48.

[20] J. G. Wardrop, "Road paper. some theoretical aspects of road traffic research," *Proceedings of the Institution of Civil Engineers*, vol. 1, no. 3, pp. 325–362, 1952. DOI: 10.1680/ipeds.1952.11259.

[21] J. d. D. Ortúzar and L. G. Willumsen, *Modelling Transport*, 4th. John Wiley & Sons, 2011, ISBN: 978-0-470-76039-0.

[22] S. J. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th. Pearson, 2020, ISBN: 978-0-13-461099-3.

[23] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, no. 7553, pp. 436–444, 2015. DOI: 10.1038/nature14539.

[24] Modal Labs, *Modal: Serverless cloud for ai/ml*, GPU compute platform for machine learning, 2024. [Online]. Available: https://modal.com/.

[25] Meta Platforms, Inc., *React: A javascript library for building user interfaces*, Frontend framework for web applications, 2024. [Online]. Available: https://react.dev/.

[26] A. Wathan and S. Schoger, *Tailwind css: A utility-first css framework*, Utility-first CSS framework, 2024. [Online]. Available: https://tailwindcss.com/.

[27] Oracle Corporation, *Mysql reference manual*, Relational database management system, 2024. [Online]. Available: https://dev.mysql.com/doc/refman/8.0/en/.

[28] Internet Engineering Task Force, *Json web token (jwt)*, RFC 7519, 2015. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc7519.

[29] N. Provos and D. Mazières, *A future-adaptable password scheme*, 1999. [Online]. Available: https://www.usenix.org/legacy/events/usenix99/provos/provos.pdf.

References

[1] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968. DOI: 10.1109/TSSC.1968.300136.

[2] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, vol. 1, no. 1, pp. 269–271, 1959. DOI: 10.1007/BF01386390.

[3] J. Redmon and A. Farhadi, "Yolov3: An incremental improvement," *arXiv preprint arXiv:1804.02767*, 2018. [Online]. Available: https://arxiv.org/abs/1804.02767.

[4] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779–788. DOI: 10.1109/CVPR.2016.91.

[5] G. Jocher, A. Chaurasia, and J. Qiu, *Ultralytics yolo*, Version 8.0, 2023. [Online]. Available: https://github.com/ultralytics/ultralytics.

[6] G. Jocher and J. Qiu, *Ultralytics yolo11*, Latest YOLO version with improved performance, 2024. [Online]. Available: https://docs.ultralytics.com/models/yolo11/.

[7] S. M. Pizer, E. P. Amburn, J. D. Austin, *et al.*, "Adaptive histogram equalization and its variations," *Computer Vision, Graphics, and Image Processing*, vol. 39, no. 3, pp. 355–368, 1987. DOI: 10.1016/S0734-189X(87)80186-X.

[8] K. Zuiderveld, "Contrast limited adaptive histogram equalization," in *Graphics Gems IV*, Academic Press Professional, Inc., 1994, pp. 474–485, ISBN: 0-12-336155-9.

[9] V. I. Levenshtein, "Binary codes capable of correcting deletions, insertions, and reversals," *Soviet Physics Doklady*, vol. 10, no. 8, pp. 707–710, 1966.

[10] M. Bachmann, *Rapidfuzz: Rapid fuzzy string matching library*, Python library for fast string matching, 2023. [Online]. Available: https://github.com/maxbachmann/RapidFuzz.

[11] A. Ronacher and Pallets Team, *Flask: A python micro web framework*, Version 3.0, 2023. [Online]. Available: https://flask.palletsprojects.com/.

[12] TomTom International BV, *Tomtom routing api documentation*, Real-time traffic and routing services, 2024. [Online]. Available: https://developer.tomtom.com/routing-api/documentation.

[13] Groq, Inc., *Groq api documentation*, Fast AI inference platform, 2024. [Online]. Available: https://console.groq.com/docs.

[14] A. A. Hagberg, D. A. Schult, and P. J. Swart, "Exploring network structure, dynamics, and function using networkx," in *Proceedings of the 7th Python in Science Conference (SciPy)*, G. Varoquaux, T. Vaught, and J. Millman, Eds., Pasadena, CA USA, 2008, pp. 11–15.

[15] G. Bradski, "The opencv library," *Dr. Dobb's Journal of Software Tools*, 2000.

[16] W. McKinney, "Data structures for statistical computing in python," in *Proceedings of the 9th Python in Science Conference*, 2010, pp. 56–61. DOI: 10.25080/Majora-92bf1922-00a.

[17] Python-Visualization Team, *Folium: Python data to leaflet.js maps*, Interactive map visualization library, 2023. [Online]. Available: https://python-visualization.github.io/folium/.

[18] V. Agafonkin, *Leaflet: An open-source javascript library for mobile-friendly interactive maps*, Version 1.9, 2023. [Online]. Available: https://leafletjs.com/.

[19] T.-Y. Lin, M. Maire, S. Belongie, *et al.*, "Microsoft coco: Common objects in context," in *European Conference on Computer Vision (ECCV)*, Springer, 2014, pp. 740–755. DOI: 10.1007/978-3-319-10602-1_48.

[20] J. G. Wardrop, "Road paper. some theoretical aspects of road traffic research," *Proceedings of the Institution of Civil Engineers*, vol. 1, no. 3, pp. 325–362, 1952. DOI: 10.1680/ipeds.1952.11259.

[21] J. d. D. Ortúzar and L. G. Willumsen, *Modelling Transport*, 4th. John Wiley & Sons, 2011, ISBN: 978-0-470-76039-0.

[22] S. J. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th. Pearson, 2020, ISBN: 978-0-13-461099-3.

[23] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, no. 7553, pp. 436–444, 2015. DOI: 10.1038/nature14539.

[24] Modal Labs, *Modal: Serverless cloud for ai/ml*, GPU compute platform for machine learning, 2024. [Online]. Available: https://modal.com/.

[25] Meta Platforms, Inc., *React: A javascript library for building user interfaces*, Frontend framework for web applications, 2024. [Online]. Available: https://react.dev/.

[26] A. Wathan and S. Schoger, *Tailwind css: A utility-first css framework*, Utility-first CSS framework, 2024. [Online]. Available: https://tailwindcss.com/.

[27] Oracle Corporation, *Mysql reference manual*, Relational database management system, 2024. [Online]. Available: https://dev.mysql.com/doc/refman/8.0/en/.

[28] Internet Engineering Task Force, *Json web token (jwt)*, RFC 7519, 2015. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc7519.

[29] N. Provos and D. Mazières, *A future-adaptable password scheme*, 1999. [Online]. Available: https://www.usenix.org/legacy/events/usenix99/provos/provos.pdf.